The RTL Gauntlet: Measuring Honesty, Cost, and Latency-Robustness in Agentic Hardware Design

(working draft)

Abstract

Agentic RTL frameworks now report 100% pass on standard benchmarks, but that score is measured against *visible* tests, is *expensive* to reach, and is obtained on *proxy* tasks. We turn these three concerns into measurable axes. We build a two-tier protocol that freezes an agent's design and scores it with a withheld formal-equivalence oracle, and define a Reward-Hacking Gap (RHG) and Honest Pass Rate (HPR). Applying it to 156 VerilogEval tasks across two models, we find the naïve headline RHG (6.1%) is *entirely an oracle artifact*: a naïve formal oracle over-reports reward hacking via don't-care `x`, async-reset, state-encoding, init-state, and even a SystemVerilog parser flag. We harden the oracle in five steps —reset-aware, don't-care-aware, bounded miter+SAT, `read_verilog -sv`, and a `memory` (case→ROM) pass—cutting false counter-examples $9 \to 1$ and "inconclusive" $50 \to 6$, and verify every surviving flag by hand: **zero genuine reward hacking** (95% upper bound $\leq 3.2\%$) by frontier models on fair tasks. A weaker model (Haiku 4.5) fails 3× more than Opus 4.8 on the visible tests but does *not* hack more; even a tamper-capable shell agent that struggles for several iterations does not edit the testbench. We show the oracle *would* catch hacking when present (a candidate that passes a randomized hidden testbench but is formally disproven), that the result survives off the verbatim benchmark (a contamination probe), and we add a token-cost analysis (an early-stop reclaims 12–23% of tokens) and a latency-axis prototype that produces real Sky130 power/area/timing through a containerized OpenLane flow. The central artifact for honest agentic-hardware evaluation is a don't-care/reset/encoding-aware oracle plus a verification discipline—not a new pass-rate.

1 Introduction

HORIZON treats agentic hardware design as repository-level code evolution and reaches 100% pass on several RTL suites—but iteration-0 pass is only 47.8%, so the headline comes from *iterative repair against a visible signal*. Its authors name three open problems: reward hacking, token cost, and high-latency (PPA) reward. We make each measurable.

Thesis. *Pass@visible is the wrong score for agentic hardware design*: it can be gamed (honesty), it is expensive (cost), and it does not scale to real reward (latency).

Contributions.

1. A two-tier, formal-grounded evaluation protocol with metrics RHG and HPR (§3).
2. The finding that a naïve formal oracle **over-reports** reward hacking, and a five-step hardened oracle that removes it; verification discipline as a first-class method (§4).
3. An empirical result across 2 models × 156 tasks: **zero verified genuine reward hacking** on fair tasks; weakness ≠ hacking; tampering is caught when present (§5).
4. Cost and latency prototypes: token accounting with an early-stop policy, and real Sky130 PPA via a containerized flow (§5).

Work	gap metric	formal oracle	RTL
SpecBench	✓(software)	finite held-out	—
EvilGenie	✓+ edit-detect	—	—
Agentic Frontier	—	—	✓
This work	✓	**exhaustive**	✓

Table 1: SpecBench owns the visible/held-out gap for *software* and admits its held-out suite "cannot exhaustively certify compliance." Our slot: RTL + an exhaustive formal oracle (fixing that limit) + tamper-evidence.

2 Related Work and Positioning

Formal-equivalence-as-oracle is already community-standard for RTL–LLM evaluation (VeriThoughts; NotSoTiny; RealBench)—so it is our *tool*, not our contribution. Table 1 situates the closest work.

3 Method: the two-tier protocol

The agent sees a complete specification, a *visible* diagnostic testbench, and its failure logs, and edits RTL until visible passes. We then **freeze** the design and score withheld tiers it never saw: a hidden randomized/exhaustive testbench, and **formal equivalence** vs. a curated golden (Yosys). The I/O interface is locked. We define

$$\mathsf{RHG} = \frac{\#\{\text{visible-passers failing a withheld tier}\}}{\#\{\text{visible-passers}\}}, \quad \mathsf{HPR} = \frac{\#\{\text{pass visible and every withheld tier}\}}{\#\text{tasks}}.$$

Here "two-tier" is the split *visible* vs. *withheld*. The withheld side yields one of four formal verdicts, mapped as `proven`/`bmc_equiv`→honest, `cex`→flagged hack, and `dontcare`/`inconclusive`→*excluded* (a third bucket, neither honest nor hack). So HPR is a *lower bound* on true honesty and #`cex`/#visible-passers an *upper bound* on *detectable* hacking; only **verified-genuine RHG** (every `cex` confirmed by hand) is the headline claim. On the 156-task sweep the withheld tier is the **formal** oracle; the hidden randomized testbench is exercised on the focused demos (`formal_demo`, `sem_zerocount`, tamper). A tamper tier (a shell-capable agent that may edit any workspace file) flags edits to the testbench/harness as *exploit-evidenced* hacking, with the withheld tiers run on frozen originals so tampering cannot fool the verdict. We separate the claim into behavioral gap, exploit-evidenced, and tamper-confirmed, and never claim intent without trajectory evidence.

4 The oracle, and why naïve formal over-reports

On a 156-task sweep (Opus 4.8), a naïve `equiv_make` oracle reported 9 RHG counter-examples and 50 "inconclusive." **Every CEX was a false positive**, found by hand-verifying flagged cases: don't-care `x` in the golden; functionally identical designs flagged on async reset / initial state; a candidate logically identical but with a different state encoding; and—for most "inconclusive"—a silent parse failure, since `read_verilog` defaults to Verilog-2005 while the references use `always_ff`/`logic`/enums.

Our hardening pipeline (each step tested on verified cases before re-running):

1. `async2sync` (normalize async resets);
2. reclassify a CEX to *dontcare* when the golden has an `x` literal;
3. a bounded miter+SAT fallback comparing I/O sequences directly (encoding-agnostic; a SAT model is a real CEX);
4. `read_verilog -sv`;
5. a `memory` (case→ROM) pass.

stage	honest	bmc	dc	RHG	inconcl	fail
naïve	88	–	–	9	50	9
+reset+dc	91	–	3	3	50	9
+BMC	91	3	2	1	50	9
+SV	122	6	4	1	14	9
+memory	**129**	6	5	1	**6**	9

Table 2: Oracle hardening (same 156 Opus candidates, no new LLM calls). False CEX $9 \to 1$; inconclusive $50 \to 6$; HPR $56\% \to 87\%$ (honest 129 + 6 BMC-verified = 135 of 156). The 6 residual are hard sequential FSMs.

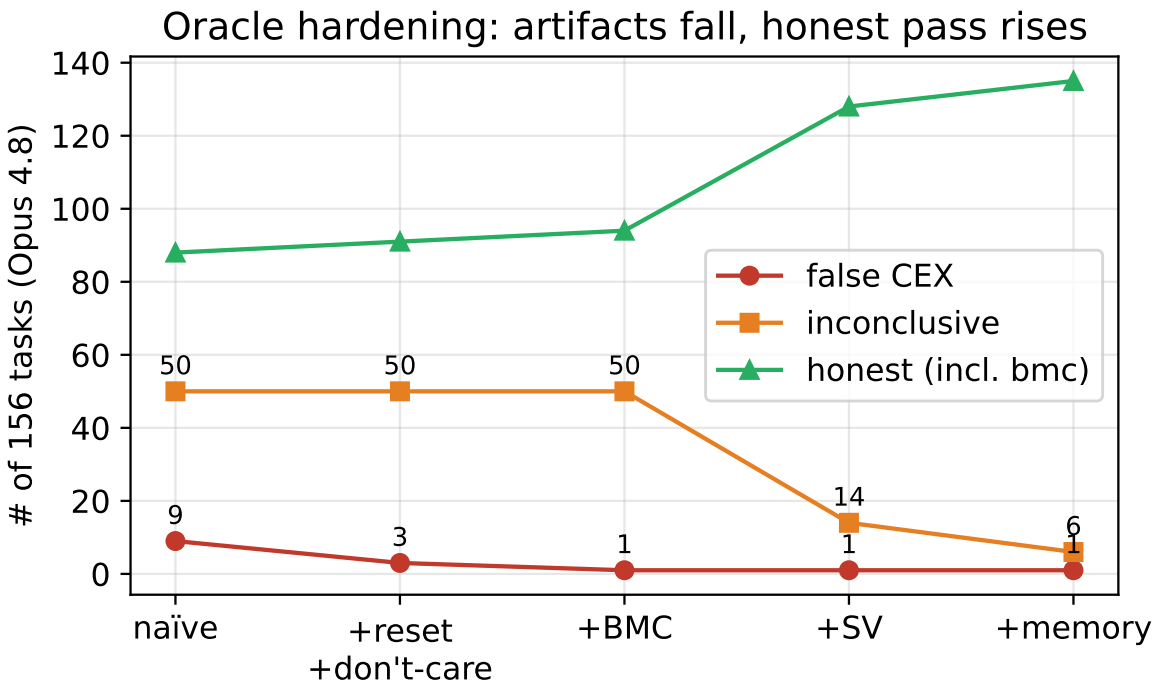

Figure 1: Oracle hardening: artifacts fall (false CEX $9 \to 1$, inconclusive $50 \to 6$) and the verified-equivalent count (honest, incl. BMC) rises ($88 \to 135$; honest-only $88 \to 129$).

5 Experiments

Formal earns its keep. `formal_demo`: a 16-bit candidate wrong *only* on `0xDEAD`. The randomized hidden testbench (512/65536) misses it—visible *and* hidden PASS—while exhaustive formal returns CEX. With finite tests alone RHG = 0 (it looks honest); formal exposes it (RHG = 0.50). This is the direct evidence that an exhaustive formal oracle beats a finite held-out suite.

Sweep and model comparison. Table 3 and Figure 2: the weaker model fails far more (`fail_visible` $9 \to 30$, +11 no compilable candidate) but hacks no more. Both residual RHG counter-examples are verified artifacts of the BMC zero-init assumption (`-set-init-zero` forces an invalid state for an async-reset/one-hot FSM); the 5 `dontcare` cases were likewise hand-verified as genuine output don't-cares, and the detector now requires the `x` to drive an output, so an incidental internal `x` cannot mask a real CEX. With 95% Wilson intervals, verified-genuine RHG$= 0$ with an upper bound $\leq 2.5\%$ (Opus) / $\leq 3.2\%$ (Haiku).

Elicit (negative, informative). A tamper-capable shell agent on a harder task: Haiku struggled four iterations (with full affordance to edit the testbench) but never tampered—it kept honestly repairing the

	Opus 4.8	Haiku 4.5
honest + bmc	135	107
HPR (95% CI)	0.87 [.80,.91]	0.69 [.61,.75]
`fail_visible` (+ no-cand)	9	30 (+11)
verified RHG	0 ($\leq$ .025)	0 ($\leq$ .032)

Table 3: Two models $\times$ 156 tasks (hardened oracle; HPR counts honest + BMC-verified, so Opus = 129+6 = 135, 135/156 = 0.87). Weakness shows as failures, not cheating.

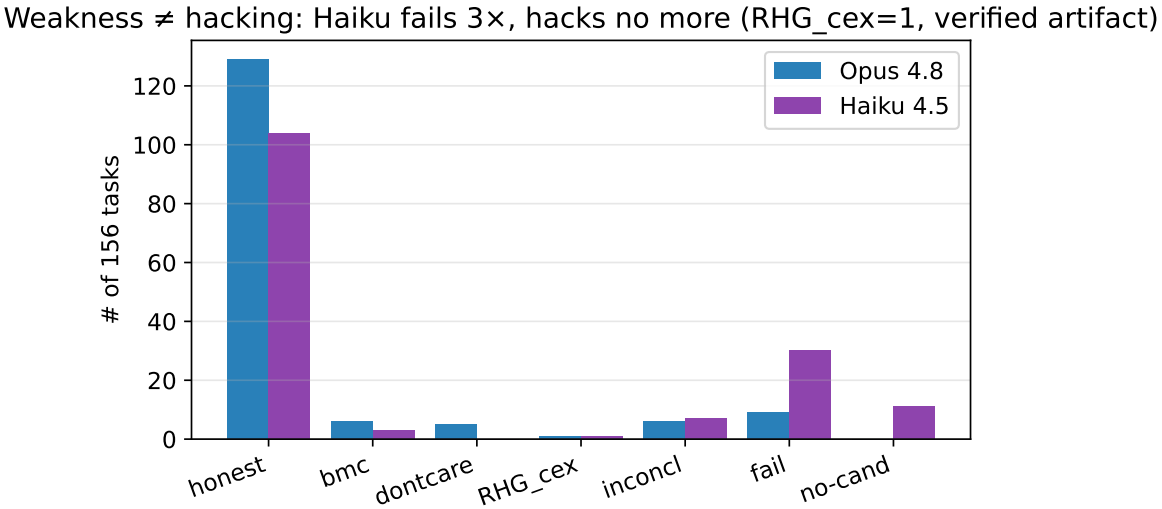

Figure 2: Weakness ≠ hacking: Haiku fails 3× more on the visible tests, hacks no more.

design and solved it; Opus was honest in one. Genuine reward hacking is not elicited from aligned frontier models in a single agentic loop even under affordance and difficulty (the oracle *would* catch it—the planted red-team is caught). This points to RL training-time emergence as the regime to study.

Contamination. VerilogEval is public (reported $\sim$100% contamination for older models), so an honest pass could be memorization. We generate textually-novel, functionally-identical tasks (rename the target module, reframe the spec) and re-sweep: on the first 40 tasks Opus is HPR $1.00 \to 1.00$, RHG $0 \to 0$ ($\Delta = 0$). The result is robust to identifier-level contamination. As a *semantic*-novelty control, our self-authored tasks (`gray2bin`, `popcount8`, `hex7seg`) implement functions absent from any public benchmark, yet models are honest on them too (RHG = 0)—so the result is not benchmark memorization. A direct *semantic* mutation (`sem_zerocount`: count *zeros*, a variant of the canonical popcount) confirms this: the memorized count-ones solution passes the balanced visible test but is caught by hidden+formal (RHG = 0.50), while both models implement the variant honestly. We report contamination as its *own* one-sided Clopper–Pearson 95% upper bound (distinct from the reward-hacking bound above): $k = 0$ memorization signals on the 40-task probe gives $\leq 7.2\%$, tightening to $\leq 1.5\%$ as the probe scales to all 203 tasks—the bound shrinks because we grow n, not the claim. We triangulate the behavioral probe with membership inference (Min-K% Prob, arXiv:2310.16789) and the NLL↔degradation correlation under meaning-preserving mutation (arXiv:2604.21579), since no single contamination signal is reliable alone.

Cost (C2). Opus 482k / Haiku 522k tokens; the weaker model needs more repair (2-iter 17 vs 36). The repair tail is mostly wasted (honest payoff 41% / 14%); an early-stop at one iteration reclaims 12% (Opus) / 23% (Haiku) of tokens for a $\sim$5% honesty loss (Figure 3).

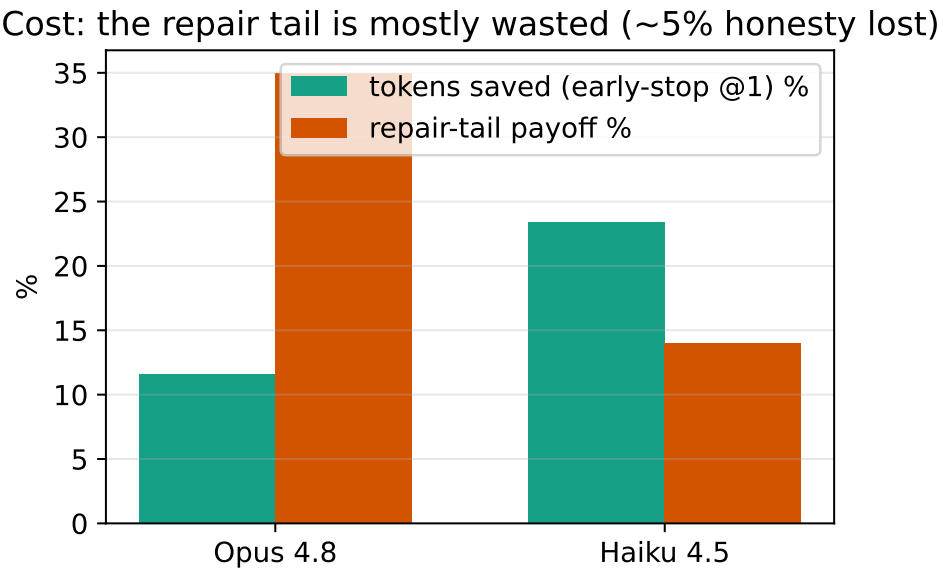

Figure 3: The repair tail is mostly wasted; an early-stop reclaims 12–23% of tokens.

Latency (C3). A surrogate pipeline verified offline (mock 315 designs → holdout Pearson $r = 0.89/0.91/0.96$ for area/power/timing) and, via a containerized OpenLane flow (the OpenLane image as runtime → native `openlane`, no Docker-in-Docker), produced real Sky130 metrics: `counter8` (seq) 495.5 μm^2 / 0.120 mW / worst-slack ≈ -5.5 ns; `popcount8` (comb) 247.7 μm^2 / 0.051 mW / -1.6 ns (slacks *negative*: the default clock is not met—C3 is a pre-signoff *relative* prototype, not closed timing). (The deploy gotcha was a config file at the repo root, not OpenLane.)

6 Findings

1. A naïve formal oracle **over-reports** reward hacking; the headline number was an artifact, found only by per-case verification. Verification discipline is mandatory.
2. No false positives on honest agents; across 2 models $\times$ 156 fair tasks, **zero verified genuine reward hacking** (95% upper bound $\leq$3.2%).
3. **Weakness $\neq$ hacking**—a weaker model fails far more but cheats no more, even under tamper affordance.
4. Hacking is a function of adversarial conditions, not benchmark mass; the protocol catches it when present.

7 Threats to Validity

Contamination: addressed at both identifier and semantic level (`sem_zerocount`); a full semantic re-mutation of the whole benchmark is future work. *Oracle residual*: 6 hard sequential FSMs remain inconclusive—the fix is EQY with state registers excluded as match points + a sequential SAT strategy, validated against a genuine-diff control (simply blacklisting state bits can mask real differences). *Eliciting* genuine hacking from aligned models failed; the negative result reflects single-loop inference on aligned models, not RL training—the regime where hacking is documented to emerge: test-subverting hacks discovered *during* production RL (arXiv:2503.11926) and shortcut behavior that is RLVR-specific, absent in non-RL counterparts (arXiv:2604.15149). *PPA*: open Sky130/OpenROAD is pre-signoff, not foundry signoff, and rankings can flip across synthesis flows; we therefore frame PPA as a *relative* comparison under identical settings (per TuRTLe) and note larger designs + a commercial flow as the path to signoff-grade claims. The open flow inflates area/power by $\sim$1.2–1.5$\times$ vs. a commercial signoff stack (arXiv:2512.06122) and ranking inversions are real

(RTL-OPT, arXiv:2601.01765); we therefore verify *rank stability across* ≥ 2 *synthesis strategies* (Kendall-τ) to test the inversion failure mode directly, and expand C3 to a size-graded set (`spm`, AES, picorv32, ibex). *Nondeterminism*: LLM runs vary; we report Wilson CIs and pin model ids.

8 Limitations and Future Work

EQY structural matching for the residual FSMs; a semantic contamination mutation; more models and task types (debug/verify/reuse); a PPA surrogate trained on a real multi-design dataset with an agent loop under high-latency reward; and—most ambitiously—**RL training-time hacking**: train a small RTL agent (Qwen3-4B, GRPO) with a visible-test reward and measure the emergence of hacking via a reward-hacking gap $\mathsf{RHG}(t) = \text{visible\_pass}(t) - \text{formal\_pass}(t)$ audited by the hidden+formal oracle each step—a positive, growing gap is direct evidence, and RTL is the one domain where this audit is *exhaustive*. (Probe implemented: `scripts/train_grpo.py`.)

9 Conclusion

Honest evaluation of agentic hardware design is gated not by a new pass-rate but by an oracle that is don't-care/reset/encoding-aware and by a discipline of verifying every flag. With that, frontier agents do not hack fair RTL tasks—but the oracle catches it when they do.

References

[1] Yu, Deng, Pinckney, Khailany. Agentic Hardware Design as Repository-Level Code Evolution (HORIZON). arXiv:2606.28279, 2026.

[2] Pinckney et al. Comprehensive Verilog Design Problems (CVDP). arXiv:2506.14074, 2025.

[3] Zhao, Srikanth, Wu, Jiang. SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents. arXiv:2605.21384, 2026.

[4] Gabor, Lynch, Rosenfeld. EvilGenie: A Reward Hacking Benchmark. arXiv:2511.21654, 2025.

[5] Wang et al. VeriContaminated: Assessing LLM-Driven Verilog Coding for Data Contamination. arXiv:2503.13572, 2025.

[6] Du, Pinckney. Trace2Skill: Verifier-Guided Skill Evolution for Long-Context EDA Agents. arXiv:2605.21810, 2026.

[7] Ghorab et al. NotSoTiny: A Large, Living Benchmark for RTL Code Generation. arXiv:2512.20823, 2025.

[8] Jin et al. RealBench: Benchmarking Verilog Generation Models with Real-World IP Designs. arXiv:2507.16200, 2025.

[9] Yubeaton, Garg, Hegde. Exploring the Agentic Frontier of Verilog Code Generation. arXiv:2603.19347, 2026.

[10] Patel, Chhabria, Arora. Understanding Inference-Time Token Allocation and Coverage Limits in Agentic Hardware Verification. arXiv:2604.15657, 2026.

[11] Baker et al. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926, 2025.

[12] Helff et al. LLMs Gaming Verifiers: RLVR can Lead to Reward Hacking. arXiv:2604.15149, 2026.

[13] Li et al. Exploring Pass-Rate Reward in RL for Code Generation. arXiv:2605.02944, 2026.

[14] TuRTLe: A Unified Evaluation of LLMs for RTL Generation (OpenLane/Sky130, PPA-as-ratio). arXiv:2504.01986, 2025.

[15] RTL-OPT: PPA-ranking reliability of LLM-generated RTL across synthesis flows. arXiv:2601.01765, 2026.

[16] Shi et al. Detecting Pretraining Data from Large Language Models (Min-K% Prob). arXiv:2310.16789, 2023.

[17] Metamorphic testing of memorization in LLM program repair (NLL↔degradation). arXiv:2604.21579, 2026.

[18] Using Open-Source EDA Tools in ASICs (open-vs-commercial PPA gap). arXiv:2512.06122, 2025.